CORNELL TECH | APPLIED MACHINE LEARNING

Human or AI?

Kaggle Competition & Report Template

Report Template: TEMPLATE

Github Repository Template: REPO

Welcome to the competition! Your task is to build a machine learning classifier that predicts whether a passage was written by a human or generated by an AI model. You will use a labeled training set, an additional unlabeled set, and a held-out Kaggle test set. The aim is not just a high leaderboard score: we want you to compare methods, design careful experiments, understand generalization, and explain what your results actually show.

You are encouraged to try several approaches. A well-tested logistic regression or support vector machine is just as legitimate as a neural network. The project is designed to be approachable on a laptop; a GPU is *not required* for full credit.

Team / members	Danny Nguyen, Ledi Wang, Rhea Pimenta
Kaggle team name	Kaggle Group 14
GitHub PR URL	https://github.com/Kickdapie/AML-Kaggle-Project-Group-14

Your data

File	Rows	Labels	Purpose
`train.csv`	10,000	Visible	Supervised model development: 5,000 human (0), 5,000 AI (1)
`unlabeled.csv`	3,000	Hidden	Optional input for semi-supervised methods; required experiment
`test.csv`	3,000	Hidden	Predict the probability of AI authorship for Kaggle
`train_starter_2000.csv`	2,000	Visible	Convenient subset of `train.csv` for quick experiments

The starter data is a subset, not 2,000 additional labeled examples. Passages are approximately 75–350 words, with mean length near 210 words.

How the test set works

The test set mixes familiar and unfamiliar writing domains and AI generators, and includes some adversarially modified AI-written passages. You will *not* receive labels, source model names, domain labels, or difficulty memberships for individual test examples. Submit a probability in $[0, 1]$ for each test ID, where 1 means AI-generated. Kaggle evaluates submissions using **area under the ROC curve (AUROC)**. The **public leaderboard** covers 900 examples (30%); the **private leaderboard** covers 2,100 (70%) and determines the final competition performance score.

1. Competition rubric and submission instructions

Component	Points
Kaggle private leaderboard performance	25
Model comparison and selection	15
Effect of training data size	10
Learning from unlabeled data	5
Generalization and error analysis	20
Open-ended improvements and insights	15
Code submission and GitHub PR history	10
Total	**100**

How to use this document. Complete the response fields and tables below; insert plots where requested. Keep your answers concise, specific, and supported by actual experiments. Points appear in brackets beside each question; no separate writing-quality grade is assigned.

How credit is awarded. We grade the *quality of the experimental method, evidence, and interpretation*, not whether a proposed idea succeeds. **Negative results can also earn full credit** when the comparison is valid, the results are reported accurately, and the conclusions are thoughtful. For each question, make clear what you tried, what you measured, and what you learned. Do not invent scores or results.

Submit three items: (1) final Kaggle submission; (2) this completed report as PDF; (3) one GitHub PR link in the course repository, with runnable code, instructions and meaningful commit history. Include the PR link on the title page. Confirm upload locations and deadline time on the course site. The template for the repository is here

Compute and models. Use models and pipelines that you can run and reproduce. Laptop-friendly training is strongly recommended; use of more expensive methods is allowed but not required for full credit. Do not use external LLM APIs to directly label or predict the competition test passages. Use only the supplied labeled dataset for supervised labels; allowed pretrained weights and AI-assisted development should be disclosed.

Validation. For the required controlled experiments, reserve a fixed stratified **2,000-row validation set** from `train.csv`, leaving up to 8,000 rows for training. Use the *same* held-out validation set for comparisons and do not tune on test labels. You may retrain your final Kaggle model on all 10,000 labeled examples after selecting your approach. Where possible, document seeds and avoid source/near-duplicate leakage.

Kaggle grading (25 points). Final performance will use the private AUROC, not leaderboard rank. The intended continuous rule is $25\,\mathrm{clip}((A - 0.50)/(T - 0.50), 0, 1)$, where A is your private AUROC and T is the announced full-credit threshold. A provisional value of $T = 0.90$ is under consideration; the instructors will confirm the fixed threshold before launch. A reference word TF-IDF and logistic regression baseline achieves approximately 0.868 private AUROC.

2. Team information and model summary

Q0.1 [0 points]. List team members and their main contributions.

[Your response: Names, NetIDs and short contribution notes]

Q0.2 [0 points]. Summarize your final approach in 3–5 sentences. What is the overall pipeline from raw text to probability?

[Your response: Brief overview]

Q0.3 [0 points]. Record your final scores (enter private score only once it is revealed).

Metric	Your score
Validation AUROC	[ ]
Kaggle public AUROC	[ ]
Kaggle private AUROC	[ ]

3. Model comparison and selection (15 points)

Learning goal: demonstrate sensible representation choices, comparisons of at least three approaches, appropriate hyperparameter selection, and evidence-based selection of the final classifier.

Q1.1 [3 points]. How did you preprocess and represent the text? Specify tokenization, normalization, word/character features or embeddings, and any feature extraction choices. Why?

[Your response: Approximately 100–150 words]

Q1.2 [5 points]. Which **three or more** modeling approaches did you try? Keep the labeled validation partition fixed across models.

Model	Text representation	Train AUROC	Val. AUROC	Train time
Model A	[Features]	[ ]	[ ]	[ ]
Model B	[Features]	[ ]	[ ]	[ ]
Model C	[Features]	[ ]	[ ]	[ ]

Q1.3 [4 points]. For at least **two** approaches, which hyperparameters and candidate values did you try? How did you select the final settings without using Kaggle test labels?

Model	Parameters/values searched	Chosen setting
Model A	[Grid/range]	[ ]
Model B	[Grid/range]	[ ]

[Your response: Explain the selection protocol, approximately 100 words]

Q1.4 [3 points]. Which approach did you choose for your final submission? Explain using validation performance, computational cost and/or stability. If you later changed the model, explain why.

[Your response: Approximately 100–150 words]

4. Effect of training data size (10 points)

Learning goal: understand the effect of the number of labeled examples on fitting and generalization. This is an empirical exercise; no formal sample-complexity theory is required.

Q2.1 [4 points]. Choose one model configuration. Train it with 500, 1,000, 2,000, 5,000 and 8,000 examples sampled from your 8,000-row training partition. Keep the same validation set and hyperparameters.

Training rows	Training AUROC	Validation AUROC	Notes (optional)
500	[ ]	[ ]	
1,000	[ ]	[ ]	
2,000	[ ]	[ ]	
5,000	[ ]	[ ]	
8,000	[ ]	[ ]	

Q2.2 [3 points]. Insert a plot of training and validation AUROC versus training-set size.

Insert learning-curve figure here

Q2.3 [3 points]. What does the curve suggest about overfitting, underfitting, and the value of more labeled examples?

[Your response: Approximately 100–150 words]

5. Learning from unlabeled data (5 points)

Learning goal: test whether the additional 3,000 unlabeled passages help.

Q3.1 [2 points]. Describe one method that uses `unlabeled.csv` (e.g., self-training, confidence-based pseudo-labeling, clustering, or feature learning). How did you prevent validation-label leakage?

[Your response: Approximately 100 words]

Q3.2 [2 points]. Compare against a supervised baseline under the same held-out validation protocol.

Method	Validation AUROC
Supervised only	[ ]
With unlabeled data	[ ]

Q3.3 [1 point]. Did it help? Give a concrete, evidence-based explanation (including possible noise in pseudo-labels).

[Your response: Approximately 100 words]

6. Generalization and error analysis (20 points)

Learning goal: characterize how well the selected model generalizes and where it fails. Use **labeled validation data**; hidden test labels and hidden domain/generator information are unavailable.

Q4.1 [5 points]. Compare training and validation AUROC for your final selected model. Is there a gap? What explanations are plausible?

[Your response: Report numbers and discuss, approximately 100 words]

Q4.2 [5 points]. Evaluate performance over at least three text-length bins (define the bins), or another observable subgroup available in your data. Explain why you chose the grouping.

Subgroup / bin	Count	Validation AUROC
Group 1	[ ]	[ ]
Group 2	[ ]	[ ]
Group 3	[ ]	[ ]

[Your response: What differences did you observe? About 100 words.]

Q4.3 [5 points]. Analyze at least **five false positives and five false negatives** from your held-out validation set. Provide example IDs, predicted probabilities, and short failure hypotheses. Do not paste full passages unless essential.

ID	Error type	P(AI)	Possible reason
[ ]	FP	[ ]	[ ]
[ ]	FP	[ ]	[ ]
[ ]	FP	[ ]	[ ]
[ ]	FP	[ ]	[ ]
[ ]	FP	[ ]	[ ]
[ ]	FN	[ ]	[ ]
[ ]	FN	[ ]	[ ]
[ ]	FN	[ ]	[ ]
[ ]	FN	[ ]	[ ]
[ ]	FN	[ ]	[ ]

Define false positive as human text predicted AI; false negative as AI text predicted human. State the decision threshold used for selecting errors (e.g., 0.5).

Q4.4 [5 points]. What recurring failure patterns did you identify, and how might you address one of them? Distinguish observed patterns from speculation.

[Your response: Approximately 150 words]

7. Open-ended improvements and insights (15 points)

Learning goal: propose and test one improvement beyond the required baseline comparisons. This can be simple: a targeted feature change, alternate regularization scheme, ensemble, different representation, augmentation, or training approach. Sophistication alone does not earn credit.

Q5.1 [3 points]. What specific question or hypothesis motivated your improvement? What weakness or opportunity did you observe?

[Your response: Approximately 75–125 words]

Q5.2 [4 points]. What exactly did you change? Describe the baseline, the new approach and what you held fixed for a fair comparison.

[Your response: Approximately 100–150 words]

Q5.3 [4 points]. What were the measured results?

Configuration	Val. AUROC	Runtime	Relevant difference
Baseline	[ ]	[ ]	[ ]
Improvement	[ ]	[ ]	[ ]

Q5.4 [4 points]. What did you learn? Did the results support the hypothesis? What limits the conclusion, and what would you try next?

[Your response: Approximately 125–175 words]

8. Code submission, reproducibility and AI assistance (10 points)

Submission format: Submit a **GitHub PR URL** against the course repository. We will inspect the final code and its development history, not just the final snapshot. Do not squash away meaningful intermediate commits before review. Template here.

Q6.1 [5 points]. Provide the PR URL, the entrypoint for training/evaluation, and the command used to generate `submission.csv`. State dependencies and random seeds.

[Your response: PR URL; commands; environment; output path]

Q6.2 [3 points]. Give two or three examples of changes you made during development. Link to relevant commits or PR discussion and briefly explain what the experiments taught you.

[Your response: Commit links and brief notes]

Q6.3 [2 points]. How did you use AI tools (such as Claude, ChatGPT or Copilot) for code, debugging, analysis or writing? What did you verify or modify yourself?

[Your response: Short, honest disclosure. Full prompt transcripts are not required.]

AI policy. You may use AI assistants for coding, debugging, explanations, brainstorming, and editing. We expect each team to exercise its own judgment, run experiments, check outputs, and understand its submitted approach. Sending the entire assignment to an AI tool and submitting its unverified output without meaningful involvement does not satisfy the objectives of this project. We will use the PR history, reproducible experiments and report together as evidence of engagement. *Commit counts and the presence of AI-generated code are not themselves grading criteria.* Do not fabricate results or retrieve hidden test labels.

Final submission checklist

☐ Final probabilities submitted to Kaggle for every `test.csv` ID.

☐ Completed report, with all required answers, tables, figures and sources, submitted as PDF.

☐ GitHub PR linked on the report title page and in the course submission portal.

☐ Code and dependencies recreate final predictions (within reasonable numerical tolerance).

☐ Relevant experiments and seed/configuration details are recorded.

☐ AI-use disclosure is complete; hidden labels were not retrieved externally.